

skills

May 20, 2026

1 ModelFlow Skills

This notebook renders the skill reference files side by side.

Files live in `modelflow/references/`: - `estimation.md` — estimator backends, constraints, initial values, Makemodel hooks - `makemodel.md` — markdown/LaTeX equation format, tags, LIST/DO templating, `replacements=`, reports

```
[1]: from pathlib import Path
from IPython.display import Markdown, display

REFS = Path.cwd() / "references"

def show_skill(name: str) -> None:
    """Render modelflow/references/<name>.md as Markdown."""
    path = REFS / f"{name}.md"
    if not path.exists():
        display(Markdown(f"**Missing:** `{path}`"))
        return
    text = path.read_text(encoding="utf-8")
    # Strip YAML frontmatter so it does not render as a thematic break.
    if text.startswith("---"):
        end = text.find("---", 3)
        if end != -1:
            text = text[end + 3:].lstrip()
    display(Markdown(text))
```

1.1 estimation.md

```
[2]: show_skill("estimation")
```

2 ModelFlow Estimation Guide

This skill is a practical reference for estimating econometric equations in ModelFlow. It covers the three estimator backends, how to specify constraints, and how to wire estimation into a `Makemodel` pipeline.

The estimation system lives in two files:

- `modelestimator_new.py` — the estimator backends and the `Eq_parent` contract
- `modelconstruct_estimation.py` — Makemodel integration (the `<estimator=...>` tag and friends)

2.1 TL;DR

```
from modelestimator_new import Estimate_ols, Estimate_nls_lmfit
```

```
# Plain OLS
```

```
est = Estimate_ols(  
    org_eq="Y = C(1) + C(2)*X + C(3)*Z",  
    input_df=df,  
    smpl=(2002, 2018),  
)  
est.coef_estimate_dict    # {'C__1': 1.23, 'C__2': 0.45, ...}  
est.org_eq_baked          # 'Y = (1.23) + (0.45)*X + (0.78)*Z'  
est.mfresult              # LSResult wrapper for HTML reports
```

```
# NLS with bounds on C(2) and a derived C(3)
```

```
est = Estimate_nls_lmfit(  
    org_eq="Y = C(1) + C(2)*X + C(3)*Z ST. C(2)=[0,1]; C(3):=C(1)*0.5",  
    input_df=df,  
    smpl=(2002, 2018),  
)
```

2.2 Choosing a backend

Backend	Use when	Equation form
<code>Estimate_ols</code>	Linear-in-parameters, no constraints	$Y = C(1) + C(2)*X + C(3)*\text{LOG}(Z)$
<code>Estimate_nls_lmfit</code>	Nonlinear-in-parameters, bounds, fixed values, or derived params	$Y = C(1) * X^{C(2)}$
<code>Estimate_nls_eviews</code>	You have EViews installed and want EViews-style NLS output	Same as <code>lmfit</code>

The `Estimate_nls` factory dispatches: `Estimate_nls(eq, solver='lmfit')` returns an `Estimate_nls_lmfit` instance.

ECM: `Estimate_nls_lmfit` defaults to `ecm=True`. The parser expects an error-correction form ($D\text{LOG}(Y) = \dots$). Set `ecm=False` if you're estimating a level equation.

2.3 Equation conventions

- Parameters are written as `C(1)`, `C(2)`, ... (EViews-style). Internally they become `C__1`, `C__2`.
- Alternate parameter prefixes are allowed via `param_names`:

```

Estimate_nls_lmfit(
    org_eq="Y = ALFA + BETA*X",
    param_names=['ALFA', 'BETA'],
    ...
)
# ALFA -> ALFA__1, BETA -> BETA__1
# Also accepts ALFA(2), BETA(3), etc.

```

- Lags: X(-1), X(-2).
- Transformations inside the equation: LOG(X), DLOG(X), DIFF(X), MOVAVG(X, n).
- The intercept is just C(1); for a no-intercept regression, omit it.

2.4 Inline constraints — the ST. clause

For nonlinear estimation via `lmfit` you can attach constraints directly to the equation, separated from the equation by `ST.` (subject to) and from each other by `;`:

```

Estimate_nls_lmfit(
    org_eq="Y = C(1) + C(2)*X + ALFA*Z ST. C(1)=[0,1]; C(2)>0; ALFA:=C(1)+C(2)",
    param_names=['ALFA'],
    input_df=df,
)

```

2.4.1 Constraint syntax reference

Form	Meaning	lmfit kwargs
NAME=[lo, hi]	Bounded between lo and hi	min=lo, max=hi
NAME > x or NAME >= x	Lower bound	min=x
NAME < x or NAME <= x	Upper bound	max=x
NAME = value	Fixed (not estimated)	value=v, vary=False
NAME ~ value	Initial value (free to change)	value=v, vary=True
NAME := expr	Derived from other params	expr='...'

Notes: - Names can be C(1) or ALFA — they are normalized just like in the equation body, so ALFA and ALFA(1) are equivalent. - **Mnemonic for = vs ~:** = is a hard pin (does not move during estimation); ~ is “approximately” — a starting value the solver is free to update. They are otherwise identical. - Multiple constraints on the same parameter accumulate: C(1) ~ 0.5; C(1) >= 0 means “start at 0.5, stay non-negative.” - The := form uses `lmfit`’s `expr`: the parameter is computed, not fitted. Reference other parameters by their normalized name (C__1, ALFA__1). - Inter-parameter inequalities like C(1) > C(2) are **not supported** — `lmfit` only handles simple bounds. Reparameterize as C(2) = C(1) - DELTA; DELTA > 0.

2.4.2 Other ways to supply constraints

Three equivalent ways:

```

# 1. Inline ST. (recommended - constraints stay with the equation)
Estimate_nls_lmfit(org_eq="Y = C(1)+C(2)*X ST. C(1)=[0,1]; C(2)>0")

```

```
# 2. As a kwarg (constraints is a normalized dict; kwarg wins on collisions)
Estimate_nls_lmfit(
    org_eq="Y = C(1)+C(2)*X",
    constraints={'C__1': {'min': 0, 'max': 1}, 'C__2': {'min': 0}},
)
```

```
# 3. Via Makemodel <constraints=...> tag (see Makemodel section)
> <estimator=nls_lmfit, constraints='C(1)=[0,1]; C(2)>0'> Y = C(1)+C(2)*X
```

2.5 Initial values — three input paths

When nonlinear estimation has trouble converging, a good initial guess matters. There are three input paths; pick whichever fits your workflow.

2.5.1 1. ~ inside the ST. clause (recommended for one-offs)

```
Estimate_nls_lmfit(
    org_eq="DLOG(Y) = C(1) + C(2)*DLOG(X) + C(3)*(LOG(Y(-1)) - LOG(X(-1))) "
    "ST. C(3) ~ -0.3; C(3) = [-1, 0]"
)
```

Reads naturally — the starting guess sits next to the bound. Best when you’re iterating on a single equation in a notebook.

2.5.2 2. default_params= kwarg (existing API, dict-based)

```
Estimate_nls_lmfit(
    org_eq="...",
    default_params={
        'C__1': {'value': 0.2},
        'C__3': {'value': -0.3, 'min': -1.0, 'max': 0.0},
    },
)
```

Useful when: - You’re driving estimation programmatically and already have a dict of starting values from a previous fit. - You want to keep the equation string clean and put all the numeric details in code.

2.5.3 3. constraints= kwarg (same shape as default_params)

```
Estimate_nls_lmfit(
    org_eq="...",
    constraints={'C__3': {'value': -0.3, 'min': -1.0, 'max': 0.0}},
)
```

Functionally identical to `default_params` for initial-value purposes. The two differ only by name and by precedence: when a parameter appears in both, `constraints` wins. Use `constraints` for things you’d express in an ST. clause, `default_params` for “default starting values I’d accept any of.”

2.5.4 Which one should I reach for?

Situation	Reach for
Iterating in a notebook, want to see initial value next to the equation	<code>ST. C(1) ~ 0.5</code>
Programmatic pipeline, starting values come from a previous fit	<code>default_params={...}</code>
<code>Makemodel</code> with many estimated equations, each needing different initial values	<code><constraints='C(1)~0.5; ...'></code> on each line
You want the initial value to be a hard fallback the solver can override only inside bounds	<code>default_params={'C__1': {'value': 0.5}}</code> + <code>ST. C(1)=[0,1]</code>

All three feed the same `params.add(name, **kwargs)` call inside `Estimate_nls_lmfit._fit()` — they merge with `default_params` lowest priority, `constraints` (inline `ST.` and `kwarg`) highest.

2.5.5 Which backends accept constraints?

- `Estimate_nls_lmfit`: **yes**
- `Estimate_ols`, `Estimate_nls_eviews`: **no** — passing constraints raises `ValueError` at construction.

2.6 Inspecting an estimator object

Every `EstimatorBackend` instance exposes:

Attribute	What it is
<code>org_eq</code>	Original equation (after uppercasing and param normalization, <code>ST.</code> clause stripped)
<code>org_eq_clean</code>	Same as <code>org_eq</code> with any <code>coef_dict</code> substitutions applied
<code>org_eq_baked</code>	Equation with estimated coefficients substituted in place (the “result” equation)
<code>coef_estimate_dict</code>	<code>{'C__1': value, ...}</code>
<code>coef_ser</code>	Same as a pandas Series
<code>regression_model</code>	Native fit result (statsmodels / lmfit / EViews spool)
<code>mfresult</code>	<code>LSResult</code> wrapper — call <code>.get_html_report()</code> for the full HTML view
<code>constraints</code>	The parsed-and-merged constraint dict
<code>c_params</code>	Sorted list of parameter placeholders found in the equation
<code>endo_var</code>	The endogenous variable name
<code>estimation_smpl</code>	The sample actually used (after NaN trimming for OLS)

2.7 Makemodel integration

Makemodel (from `modelconstruct_estimation.py`) accepts a *markdown* model text where each equation lives on a line beginning with `>` and continuation lines begin with `>>`. Tags such as `<estimator=...>` attach to the equation. The parser extracts these lines (ignoring surrounding prose), runs the estimator for each tagged equation, and bakes the estimated coefficients into the equation before normalization.

2.7.1 Markdown equation format

- `>` starts an equation. No FRML keyword, no `$` terminator.
- `>>` continues the previous equation on the next line (the two are joined with a space).
- Tags written inside `< ... >` attach metadata to the equation; `<estimator=...>` is the most important one.
- Lines not starting with `>` are prose — they are ignored by the model parser, so the file can mix documentation with equations.

2.7.2 Basic usage

```
from modelconstruct_estimation import Makemodel
from modelestimators_new import Estimate_nls

ls = Estimate_nls.with_defaults(input_df=df, smpl=(2002, 2018))

mm = Makemodel("""
Some prose explaining the model.

> <estimator=ls> DLOG(Y) = C(1) + C(2)*DLOG(X)
>> + C(3)*(LOG(Y(-1)) - LOG(X(-1)))
> <ident> Z = Y * 0.5
""")
mmmodel = mm.mmmodel          # ready-to-solve modelflow model
mm.estimate_report()          # writes makemodel_estimation_report.html
```

2.7.3 Supported tags

Tag	Meaning	Example
<code><estimator=ls></code>	Use the local-namespace <code>ls</code> object as the estimator class	<code><estimator=ls></code>
<code><estimator=nls_lmfit></code>	Use a registered estimator class	<code><estimator=nls_lmfit></code>
<code><est=ls></code>	Short alias for <code><estimator=...></code>	<code><est=ls></code>
<code><smpl=2002 2018></code>	Override sample for this equation	<code><smpl=2010 2020></code>
<code><caption='... '></code>	Override caption shown in the report	<code><caption='Inflation eq'></code>

Tag	Meaning	Example
<code><constraints='... '></code>	Append constraints (same syntax as ST. clause)	<code><constraints='C(1)=[0,1]; C(2)>0'></code>
<code><ident></code>	Identity (no estimation) — equation is a definition	<code><ident> Z = Y * 0.5</code>

2.7.4 Constraints from a tag

These three forms produce the same estimator state:

A. Inline ST.

```
> <estimator=ls> Y = C(1)+C(2)*X ST. C(1)=[0,1]
```

B. Tag

```
> <estimator=ls, constraints='C(1)=[0,1] '> Y = C(1)+C(2)*X
```

C. Both - merged together (tag wins on key collisions)

```
> <estimator=ls, constraints='C(2)>0'> Y = C(1)+C(2)*X ST. C(1)=[0,1]
```

2.7.5 Templating with replacements=

Makemodel accepts a `replacements=` kwarg that applies one or more (old, new) string substitutions to the model text *before* parsing. This is a plain `str.replace` — no regex — and it operates on the raw formula text.

Two common uses:

1. Keeping the model parsimonious. Write the equation once with a short placeholder, substitute in the real variable name at construction time:

```
template = """
> <estimator=ls> LOG(CONS) = C(1) + C(2)*LOG(__INCOME)
"""
```

```
mm = Makemodel(
    template,
    replacements=[('__INCOME', 'GDP_REAL_DISP')],
    input_df=df,
    estimator=ls,
)
```

2. Same template, many entities (banks, countries, sectors). Write the model once with a sentinel like `__dim`, then loop:

```
template = """
> <estimator=ls> LOG(LOANS__dim) = C(1) + C(2)*LOG(GDP__dim)
> <ident> NPL__dim = LOANS__dim * NPL_RATIO__dim
"""
```

```
bank_models = {
```

```

bank: Makemodel(
    template,
    replacements=[('__dim', f'_{bank}')],
    input_df=df,
    estimator=ls,
)
for bank in ['IB', 'SOREN', 'MARIE']
}
# bank_models['IB'].mmodel has LOANS_IB, GDP_IB, ...
# bank_models['SOREN'].mmodel has LOANS_SOREN, GDP_SOREN, ...

```

Accepted shapes:

```

replacements=('OLD', 'NEW')           # single pair
replacements=[('A', 'B'), ('C', 'D')] # list, applied in order
replacements=[]                       # no-op (default)

```

Notes:

- Substitution is order-sensitive: later pairs see the output of earlier ones, so make the sentinels unique enough that early replacements don't collide with later ones (the leading `__` convention helps).
- `replacements=` substitutes blindly — it will also touch any non-equation prose between the `>` lines. Keep sentinels (`__dim`, `__BANK_PLACEHOLDER`) distinctive so this doesn't matter in practice.
- Compared with `LIST`/`TLIST` blocks: `LIST` is the right tool when the *same model instance* needs to enumerate sublists at compile time (e.g. one set of equations that loops over all banks inside a single solve). `replacements=` is the right tool when you want *one model instance per entity*, each solvable independently. Use `LIST` for “all banks in one model”; use `replacements=` for “one model per bank.”

2.8 Recipes

2.8.1 Bound the speed-of-adjustment coefficient in an ECM

```

Estimate_nls_lmfit(
    org_eq=(
        "DLOG(Y) = C(1) + C(2)*DLOG(X) + C(3)*(LOG(Y(-1)) - LOG(X(-1))) "
        "ST. C(3) ~ -0.3; C(3) = [-1, 0]"
    ),
    input_df=df,
)
# Starts at -0.3 (a typical ECM speed-of-adjustment), stays in (-1, 0).

```

2.8.2 Pin one coefficient, estimate the rest

```

Estimate_nls_lmfit(
    org_eq="Y = C(1) + C(2)*X + C(3)*Z ST. C(3)=0.5",
    input_df=df,
)
# C(3) is held at 0.5 (vary=False) while C(1) and C(2) are estimated.

```


2.8.3 Tie two coefficients together

```
Estimate_nls_lmfit(
    org_eq="Y = C(1)*X + C(2)*Z + C(3)*W ST. C(3):=C(1)+C(2)",
    input_df=df,
)
# C(3) is computed as C(1)+C(2) - not a free parameter.
```

2.8.4 Estimate one equation in a larger model

```
mm = Makemodel("""
A small consumption model.

> <estimator=ls> LOG(C) = C(1) + C(2)*LOG(Y) + C(3)*R
> <ident>          S = Y - C
> <ident>          I = S - DEF
""", input_df=df, estimator=ls)
```

Only the first equation is estimated; the others remain as identities.

2.9 Authoring a new estimator backend

Subclass `EstimatorBackend` and implement two methods:

```
@dataclass
class Estimate_gls(EstimatorBackend):
    sigma: Optional[np.ndarray] = None
    ecm: bool = False

    def _fit(self) -> Tuple[Any, Dict[str, float]]:
        df = self.estimate_df
        y, X = df["LHS"], df.drop(columns=["LHS"])
        result = sm.GLS(y, X, sigma=self.sigma).fit()
        return result, self._map_statsmodels_params(result.params.to_dict())

    def _render_summary_html(self) -> str:
        return self.regression_model.summary().as_html()
```

That's it. Sample handling, A/F plots, `org_eq_baked` substitution, `LSResult` HTML, and `Makemodel` integration all work automatically.

To opt into constraints: set `_supports_constraints: ClassVar[bool] = True` and use `self.constraints` in `_fit()`.

To reject equation forms the backend can't handle: override `_validate_equation()` and raise `ValueError` with a helpful message.

2.10 Common pitfalls

- **Estimate_ols: “no usable rows in sample”** — lags or LOG transforms produced NaN/inf rows. Either widen the sample, fix the data, or remove the problematic transformation.

- **Estimate_ols: “no parameters to estimate”** — the equation has no `C(...)` placeholders. There’s nothing to fit. Add at least `C(1)`.
- **Estimate_ols: “no intercept” warning** — fitting through the origin. Usually unintentional; add `C(1)` if you want an intercept.
- **ValueError: ... does not support constraints** — you supplied constraints to OLS or EViews. Switch to `Estimate_nls_lmfit` or remove the `ST.` clause / `constraints=` kwarg.
- **ValueError: constraints reference unknown parameters** — typo: e.g. `ST. C(7)=[0,1]` when the equation only uses `C(1)..C(3)`. Either fix the constraint or add the missing parameter to the equation.
- **lmfit doesn’t converge / coefficients blow up** — supply better initial values via `default_params={'C__1': {'value': 0.5}, ...}` or tighten bounds with the `ST.` clause.

2.11 Reports

- **Single estimator:** `est.mfresult.get_html_report()` returns a full HTML report (regression summary + equation listing + actual-vs-fitted plot). Use `est.mfresult.show()` to open in a browser.
- **From a Makemodel:** `mm.estimate_report(path='html', filename='out.html', open_file=True)` exports all estimations in the model.

2.12 Related modules

- `modelnormalize.py` — turns `DLOG(Y) = ...` into a solvable form; called by `Eq_parent`
- `modelclass.py` — the core `model` class; estimated equations flow into this
- `modelmf.py` — the `.mfcalc` pandas accessor used by OLS to materialize regressors
- `modelestimation.py` — *older* estimation module; superseded by `modelestimator_new.py`. Avoid for new work.

2.13 makemodel.md

```
[3]: show_skill("makemodel")
```

3 Makemodel Guide

`Makemodel` is the high-level entry point for authoring `ModelFlow` models. It takes a *markdown*-style text containing prose mixed with equations, expands `LIST/DO` templates, runs any tagged estimations, normalizes the result, and exposes a ready-to-solve `model` object.

This skill covers the authoring workflow end-to-end. For deeper detail on estimator backends (OLS, `lmfit`, EViews, constraints), see the **estimation** skill.

`Makemodel` lives in `modelconstruct_estimation.py` (and a non-estimation flavor in `modelconstruct.py` — use the `_estimation` one unless you specifically don’t want estimation support).

3.1 TL;DR

```
from modelconstruct_estimation import Makemodel
from modelestimators_new import Estimate_nls

ls = Estimate_nls.with_defaults(input_df=df, smpl=(2002, 2018))

mm = Makemodel("""
A small consumption model.

> <estimator=ls> LOG(C) = C(1) + C(2)*LOG(Y) + C(3)*R
> <ident>          S = Y - C
> <ident>          I = S - DEF
""", input_df=df, estimator=ls)

mm.mmodel          # ready-to-solve modelflow model
mm.show            # print normalized FRMLs
mm.draw            # visualize the dependency graph
mm.estimation_report() # write makemodel_estimation_report.html
```

3.2 Equation input formats

Makemodel accepts two input formats — pick whichever is more natural for the situation. Both can be mixed inside the same text.

Format	When to use
> markdown	Fast authoring, Jupyter cells, code-style notation
LaTeX	Equations meant to be rendered in a paper or report; complex algebra; clear typographic separation

3.2.1 Format 1 — Markdown (> lines)

Makemodel reads markdown text where:

Line prefix	Meaning
>	An equation. No FRML keyword, no \$ terminator.
>>	Continues the previous equation (the two lines are joined with a space).
>list ...	A LIST or TLIST definition (see Templating section).
anything else	Prose. Ignored by the parser — you can mix documentation and equations freely.

Tags written inside < ... > attach metadata to the equation. <estimator=...> triggers estimation; <ident> declares an identity; others (<smpl>, <caption>, <constraints>, <endo>, <stoc>,

etc.) modify behavior.

Continuation example

A long equation broken over several lines:

```
> <estimator=ls> DLOG(Y) = C(1) + C(2)*DLOG(X)
>>                  + C(3)*DLOG(Z)
>>                  + C(4)*(LOG(Y(-1)) - LOG(X(-1)))
```

is equivalent to

```
> <estimator=ls> DLOG(Y) = C(1) + C(2)*DLOG(X) + C(3)*DLOG(Z) + C(4)*(LOG(Y(-1)) - LOG(X(-1)))
```

3.2.2 Format 2 — LaTeX (`\begin{equation}` blocks)

Equations can also be written as standard LaTeX equation environments. Two rules:

- **Every equation must have a `\label{eq:...}`.** Equations without a label are skipped by the parser — this is deliberate so you can put display-only math in the document.
- **Tags use the LaTeX comment marker.** Write `% @<tag>` on its own line inside the block, instead of the `<tag>` form used in `>` markdown.

```
\begin{equation}
\label{eq:consumption}
% @<estimator=ls>
\log(C_t) = C(1) + C(2) \cdot \log(Y_t) + C(3) \cdot R_t
\end{equation}
```

The parser translates the LaTeX body to ModelFlow notation through `latex_to_doable`. The most useful conversions:

LaTeX	Becomes	Notes
<code>_t</code>	(stripped)	Current period — the <code>t</code> index drops out
<code>_{t-1}</code>	<code>(-1)</code>	Lag operator
<code>_{t+1}</code>	<code>(+1)</code>	Lead operator
<code>^{x}</code>	<code>__{x}</code>	Templating dimension (drives <code>doable</code> expansion)
<code>^{x,y}</code>	<code>__{x}__{y}</code>	Multi-dimensional templating (up to 4 dims supported)
<code>\Delta X_t</code>	<code>diff(X)</code>	First difference
<code>\sum_{X}(expr)</code>	<code>sum(X, expr)</code>	Sum across a LIST named <code>X</code>
<code>\sum_{X=k}(expr)</code>	<code>sum(X k=1, expr)</code>	Sum starting at sublist position 1
<code>\max_{X}(expr) /</code> <code>\min_{X}(expr)</code>	<code>lmax(X, expr) / lmin(X,</code> <code>expr)</code>	List min/max
<code>\frac{a}{b}</code>	<code>(a)/(b)</code>	Fractions are flattened
<code>\sqrt{x}</code>	<code>sqrt(x)</code>	
<code>x^y</code>	<code>x**y</code>	Power (only when not used as a dim)

LaTeX	Becomes	Notes
<code>\Phi(z) / \Phi^{-1}(z)</code>	<code>NORM.CDF(z) / NORM.PDF(z)</code>	Normal CDF / inverse
<code>\rho, \alpha, \beta, \tau,</code> <code>\sigma, \exp</code>	<code>rho, alpha, beta, ...</code>	Common Greek letters
<code>\times, \cdot</code>	<code>*</code>	Multiplication
<code>\forall</code>	<code>[agegroup=working]</code>	Sublist filter (BLL doable [...])
<code>[agegroup=working]</code> <code>\text{[banks=selected]}</code>	<code>[banks=selected]</code>	Same — alternative LaTeX wrapping
<code>\left, \right, \;, \,, \big,</code> <code>\nonumber</code>	(stripped)	Whitespace/sizing markup
<code>\begin{aligned} ...</code> <code>\end{aligned},</code> <code>\begin{split} ...</code> <code>\end{split}</code>	(stripped)	Multi-line environments are flattened

If the body still contains a `\` after translation, `Makemodel` raises `ModelSpecificationError` telling you which fragment didn't translate. Add a regex or fix the LaTeX accordingly.

LIST definitions inside LaTeX LISTs can be written in inline math (`$ ... $`) so they render correctly in the document:

```
$List \; agegroup = \{16, 17, 18, 19, 20, 99, 100\}$

\begin{equation}
\label{eq:population_dynamics}
\forall [agegroup=middle] \; Population^{agegroup}_t =
    Population^{agegroup-1}_{t-1} - Dead^{agegroup-1}_{t-1}
\end{equation}
```

The parser harvests every `$List ... $` block in the text before processing equations, regardless of where they appear.

LaTeX with `<estimator=...>` The `% @<...>` tag accepts the same content as the `>`-markdown tag form, so estimation works identically:

```
\begin{equation}
\label{eq:phillips}
% @<estimator=nls_lmfit, constraints='C(2)>0'>
\Delta \log(P_t) = C(1) + C(2) \cdot UR_t + C(3) \cdot \Delta \log(P_{t-1})
\end{equation}
```

Mixing formats You can mix both formats in one `Makemodel` call. A typical use is LaTeX for the main estimated equation (so it renders nicely in a paper) and `>` markdown for accounting identities:

The behavioral equation:

```

\begin{equation}
\label{eq:consumption}
% @<estimator=ls>
\log(C_t) = C(1) + C(2) \log(Y_t)
\end{equation}

```

Identity definitions:

```

> S = Y - C
> I = S - DEF

```

3.3 Tags reference

Tag	Meaning
<ident>	Identity (definition). The default if no other tag is present.
<estimator=NAME>	Run estimator NAME on this equation; bake the estimated coefficients in.
<est=NAME>	Short alias for <estimator=NAME>.
<smpl=START END>	Override the estimation sample for this equation.
<caption='... '>	Override the caption shown in the report.
<constraints='... '>	Constraints on estimated parameters (same syntax as inline ST.).
<stoc>	Stochastic equation — add an add-factor variable so the model can be calibrated to history.
<exo>	Exogenize the LHS — add a fixable add-factor variable.
<fit>	Generate a fitted-value variant alongside the equation.
<endo=NAME>	Override which variable on the equation is endogenous.
<endo_lhs=FALSE>	The LHS is not the endogenous variable (use with <endo=...>).
<implicit>	Equation is implicit (cannot be solved by simple substitution).

Multiple tags can appear in one < ... > block, separated by commas:

```

> <estimator=ls, smpl=2010 2018, constraints='C(2)>0'> DLOG(Y) = C(1) + C(2)*DLOG(X)

```

In a LaTeX equation, tags use the comment-marker form % @<...> on its own line inside the block — see the LaTeX section above.

3.4 Templating

3.4.1 LIST blocks — enumerated sublists

A LIST defines a named set with optional sublists. The pattern is `LIST name = sublist : values / sublist : values / ...`. Inside equations, `{name}` expands to each value and `{sublist}` expands to the parallel value from another sublist.

```
>list banks = banks : IB      SOREN  MARIE /
>>          country : DENMARK SWEDEN DENMARK /
>>          selected : 1      1      0
```

```
>list sectors = sectors : NFC SME HH
```

Loss equation, expanded for every bank × sector:

```
> doable LOSS__{banks}__{sectors} = HOLDING__{banks}__{sectors} * PD__{banks}__{sectors}
```

After expansion, Makemodel produces nine equations (3 banks × 3 sectors), e.g. `LOSS__IB__NFC = HOLDING__IB__NFC * PD__IB__NFC`.

3.4.2 TLIST — transposed list

When the natural reading is columns-by-rows rather than rows-by-columns, TLIST accepts the transposed form and rewrites it internally to a regular LIST:

```
>tlist banks = banks country selected /
>>          IB      DENMARK 1 /
>>          SOREN SWEDEN  1 /
>>          MARIE DENMARK 0
```

is equivalent to the LIST example above.

3.4.3 DO / ENDDO — explicit loops

For control over which list to loop and which conditions apply:

```
> do banks
>   £ comment: equations for bank {banks} in {country}
>   x_{banks} = 42
> enddo
```

The £ symbol is the BLL comment character (comments are preserved into the FRML output but not parsed).

3.4.4 DOABLE — single-line DO

doable (also written DOABLE) is a one-line loop over every LIST appearing inside {...}:

```
> doable LOSS__{banks}__{sectors} = HOLDING__{banks}__{sectors} * PD__{banks}__{sectors}
```

Equivalent to nested `do banks / do sectors / ... / enddo / enddo` but more compact.

3.4.5 Filtering with [sublist=value]

You can restrict a doable to a subset using a sublist condition:

```
> doable [banks selected=1] x_{banks} = 42
```

Only generates equations for banks where `selected = 1`.

3.4.6 replacements= — Python-side string substitution

Apply one or more (old, new) string substitutions to the model text *before* parsing. Useful for two patterns:

Parsimony — write a placeholder once, substitute the real name at construction:

```
template = """
> <estimator=ls> LOG(CONS) = C(1) + C(2)*LOG(__INCOME)
"""
```

```
mm = Makemodel(
    template,
    replacements=[('__INCOME', 'GDP_REAL_DISP')],
    input_df=df,
    estimator=ls,
)
```

Same template, many entities — write the model once, then loop:

```
template = """
> <estimator=ls> LOG(LOANS__dim) = C(1) + C(2)*LOG(GDP__dim)
> <ident>      NPL__dim = LOANS__dim * NPL_RATIO__dim
"""
```

```
bank_models = {
    bank: Makemodel(
        template,
        replacements=[('__dim', f'_{bank}')],
        input_df=df,
        estimator=ls,
    )
    for bank in ['IB', 'SOREN', 'MARIE']
}
```

Accepted shapes:

```
replacements=('OLD', 'NEW')           # single pair
replacements=[('A', 'B'), ('C', 'D')] # list, applied in order
replacements=[]                       # no-op (default)
```

Notes: - Plain `str.replace` — no regex. Make sentinels distinctive (`__dim`, `__INCOME`, ...) so they don't collide. - Substitution is order-sensitive: later pairs see earlier output. - Operates on the entire model text — including prose between `>` lines. - Use `LIST` for “all banks in one model”; use `replacements=` for “one model per bank.”

3.5 Estimation integration

Any equation tagged with `<estimator=NAME>` is estimated during `Makemodel.__post_init__`. The estimated coefficients are baked into the equation before normalization, so the resulting `mmodel` has numeric coefficients in place of `C(1)`, `C(2)`, ...

3.5.1 Basic estimation

```
from modelconstruct_estimation import Makemodel
from modelestimators_new import Estimate_nls

# Factory captures shared defaults
ls = Estimate_nls.with_defaults(input_df=df, smpl=(2002, 2018))

mm = Makemodel("""
> <estimator=ls> DLOG(Y) = C(1) + C(2)*DLOG(X) + C(3)*(LOG(Y(-1)) - LOG(X(-1)))
> <ident>          Z = Y * 0.5
""", input_df=df, estimator=ls)
```

The estimator name `ls` is resolved from the caller's namespace (so `Estimate_nls.with_defaults(...)` stored in a local variable just works), or from an explicit `estimator_classes={'ls': ...}` mapping if you prefer.

3.5.2 Estimation-related Makemodel kwargs

Kwarg	Purpose
<code>input_df=</code>	DataFrame fed to each estimator. Each estimator may override via its own <code>input_df</code> .
<code>estimator=</code>	Default estimator for <code><estimator></code> flags with no value.
<code>smpl=</code>	Default estimation sample. <code><smpl=START END></code> per equation overrides.
<code>estimator_kwargs={}</code>	Shared kwargs (e.g. <code>caption</code> , <code>add_add_factor</code>) passed to every estimator.
<code>estimator_classes={}</code>	Explicit name $\rightarrow$ class mapping (alternative to caller-namespace resolution).
<code>estimator_namespace={}</code>	Namespace dict for resolving <code><estimator=ls></code> -style names.

3.5.3 Constraints on estimated parameters

Two equivalent forms:

Inline ST. clause (constraints stay with the equation):

```
> <estimator=ls> Y = C(1) + C(2)*X ST. C(1)~0.5; C(2)>0; C(2)<=1
```

<constraints=...> tag (separate from the equation):

```
> <estimator=ls, constraints='C(1)~0.5; C(2)>0; C(2)<=1'> Y = C(1) + C(2)*X
```

Recognized constraint forms (full reference in the **estimation** skill):

Form	Meaning
NAME=[lo, hi]	Bounded between lo and hi
NAME > x / >= x	Lower bound
NAME < x / <= x	Upper bound
NAME = value	Fixed (not estimated)
NAME ~ value	Initial value (free to change)
NAME := expr	Derived from other params

Constraints work with `Estimate_nls_lmfit`. OLS and EViews raise an error if constraints are supplied.

3.5.4 Inspecting estimations

After construction:

```
mm.estimated_records      # list of dicts, one per estimated equation
mm.estimated_report()    # writes HTML report (all estimations)
mm.markdown_with_estimation # original markdown + inline coefficient tables
mm.render_est            # interactive display of the above
```

Each `estimated_records` entry contains:

```
{
    'equation_index': int,          # position in the original model
    'frmlname': str,              # original FRML name flag
    'estimator': str,             # display name of the estimator
    'smpl': tuple,                # sample used
    'original_expression': str,    # before baking
    'baked_expression': str,      # with coefficients substituted
    'estimator_object': EstimatorBackend, # the live estimator
}
```

Use `record['estimator_object'].mfresult.get_html_report()` for the full single-equation HTML view.

3.6 Outputs and useful attributes

Attribute	What it is
<code>mm.model</code>	The full ready-to-solve <code>model</code> (cached). Use this for forecasting/simulation.
<code>mm.clean_model</code>	A <code>model</code> containing only the core equations (no add-factor or fitted variants).
<code>mm.add_model</code>	A <code>model</code> that <i>calculates</i> add factors from historic data.
<code>mm.normal_frml</code>	The fully normalized FRML text.

Attribute	What it is
<code>mm.clean_frml</code>	Normalized FRMLs with ADD/EXO/FIT options stripped.
<code>mm.show</code>	Prints <code>normal_frml</code> .
<code>mm.draw</code>	Plots the dependency graph (uses <code>modelnet</code>).
<code>mm.modellist</code>	Dict of parsed LIST definitions.
<code>mm.showlists</code>	Pretty-prints the LIST definitions.
<code>mm.render</code>	Renders the markdown source (equations + prose) in a notebook.
<code>mm.render_est</code>	Same as <code>render</code> , plus estimation tables inline.
<code>mm.estimation_records</code>	List of estimation result records (see above).

3.7 Aligning to historic data — `init_addfactors`

For models with `<stoc>` or `<exo>` tags, ModelFlow generates add-factor variables (`*_A`). `init_addfactors` computes the add factors needed to make the model exactly reproduce a historic dataframe:

```
aligned_df = mm.init_addfactors(df, start=2002, end=2020, show=True, check=True)
# aligned_df has the add-factor columns filled in
# show=True prints them; check=True re-simulates to verify
```

After this, `mm.mmodel(aligned_df, ...)` reproduces history exactly, and any change to a variable produces a counter-factual.

3.8 Combining models — `+`

Two `Makemodel` instances can be concatenated:

```
core    = Makemodel(core_equations)
shocks  = Makemodel(shock_equations)
full    = core + shocks
full.mmodel # contains both blocks
```

LIST definitions are merged. Useful for keeping a base model in one cell and scenario overrides in another.

3.9 Recipes

3.9.1 Quick identity-only model

```
mm = Makemodel("""
> Y = C + I + G + X - M
> S = Y - T
> DEF = G - T
""")
mm.mmodel
```

(No `<ident>` needed — equations without tags default to identity.)

3.9.2 One estimated equation in a larger model

```
mm = Makemodel("""
Consumption is estimated; the other equations are identities.

> <estimator=ls> LOG(C) = C(1) + C(2)*LOG(Y) + C(3)*R
> S = Y - C
> I = S - DEF
""", input_df=df, estimator=ls)
```

3.9.3 Mixed estimators

```
mm = Makemodel("""
> <estimator=nls_lmfit> DLOG(C) = C(1) + C(2)*DLOG(Y) + C(3)*(LOG(C(-1)) - LOG(Y(-1))) ST. C(3)
> <estimator=ols>      LOG(I) = C(1) + C(2)*LOG(Y) + C(3)*R
> <ident>              S = Y - C
""", input_df=df)
```

3.9.4 Stochastic equation (add-factor variant)

```
mm = Makemodel("""
> <estimator=ls, stoc> LOG(C) = C(1) + C(2)*LOG(Y)
""", input_df=df, estimator=ls)

mm.init_addfactors(df_history) # populates LOG_C_A so model reproduces history
```

3.9.5 Bank-by-bank credit-loss model

```
template = """
> <estimator=ls> LOG(LOSS__bank) = C(1) + C(2)*LOG(GDP__bank) + C(3)*UR__bank
> <ident>      NPL__bank = LOSS__bank * NPL_FACTOR__bank
"""

models = {
    b: Makemodel(template, replacements=[('__bank', f'_{b}')], input_df=df, estimator=ls)
    for b in ['IB', 'SOREN', 'MARIE']
}
```

3.9.6 Many sectors inside a single model

```
mm = Makemodel("""
>list sectors = sectors : NFC SME HH

> doable <estimator=ls, stoc> LOG(LOSS__{sectors}) = C(1) + C(2)*LOG(GDP)
""", input_df=df, estimator=ls)
# Three estimated equations: LOSS_NFC, LOSS_SME, LOSS_HH
```

3.9.7 LaTeX-authored model

```
mm = Makemodel(r"""
A small Phillips-curve model.

$List \; lags = \{1, 2, 3, 4\}$

\begin{equation}
\label{eq:phillips}
% @<estimator=nls_lmfit>
\Delta \log(P_t) = C(1) + C(2) \cdot UR_t + C(3) \cdot \Delta \log(P_{t-1})
\end{equation}

\begin{equation}
\label{eq:unemployment}
% @<ident>
UR_t = 100 \cdot (1 - E_t / L_t)
\end{equation}
""", input_df=df)
```

3.9.8 Same equation, two notations

These two `Makemodel` calls produce identical models:

```
mm_md = Makemodel("""
> <estimator=ls> LOG(C) = C(1) + C(2)*LOG(Y) + C(3)*LOG(C(-1))
""", input_df=df, estimator=ls)

mm_tex = Makemodel(r"""
\begin{equation}
\label{eq:consumption}
% @<estimator=ls>
\log(C_t) = C(1) + C(2) \log(Y_t) + C(3) \log(C_{t-1})
\end{equation}
""", input_df=df, estimator=ls)
```

3.10 Common pitfalls

- **<estimator=ls> not found** — make sure `ls` is in the caller's scope, or pass `estimator_classes={'ls': ls} / estimator_namespace=locals()`.
- **Equations don't expand inside DO blocks** — check that you're using `{name}` (with braces) not `name`.
- **replacements= matched too much** — your sentinel collided with a real token. Use distinctive prefixes (`__bank`, not `BANK`).
- **replacements= is plural** — the kwarg name is `replacements`, not `replace` (a common typo).
- **Estimator can't take constraints** — OLS and EViews backends raise on any `ST.` clause or `<constraints=...>` tag. Switch to `Estimate_nls_lmfit`.
- **Estimation sample shrinks unexpectedly** — OLS drops NaN/inf rows from lags and

LOG() transforms; the actual sample is reported in `estimator_obj.estimate_smpl`.

- **mm.mmodel is cached** — re-running estimation requires constructing a new `Makemodel`. The cached model property doesn't refresh automatically.
- **LaTeX equation skipped silently** — every equation environment needs `\label{eq:...}`. Equations without a label are treated as display-only and dropped by the parser. Add a label even if you don't reference it elsewhere.
- **ModelSpecificationError: Some LaTeX has survived** — a LaTeX construct didn't translate (typically a Greek letter or operator not in the conversion table). Either rewrite that part in plain notation, or extend the translation tables in `latex_to_doable` (in `modelconstruct_estimation.py`).
- **Raw strings for LaTeX** — pass LaTeX-containing model text as `r"""..."""` so backslashes survive intact: `\Delta` would otherwise be interpreted by Python.

3.11 Related modules and skills

- **estimation** skill — deeper detail on the estimator backends (OLS / `lmfit` / `EViews`), constraint forms, initial-value strategies, and authoring new backends.
- `modelconstruct.py` — same dataclass without estimation hooks. Use the `_estimation` variant unless you specifically want the smaller surface.
- `modelclass.py` — the underlying `model` class that `mm.mmodel` returns.
- `modelnormalize.py` — handles `DLOG/DIFF/MOVAVG` rewriting; called from `Makemodel.__post_init__`.
- `modelmanipulation.py` — `tofrml`, `dounloop`, `sumunroll`; the `LIST/DO` expansion engine.
- `modelpattern.py` — `kw_frml_name` extracts tag values like `<estimator=NAME>`.